GUÍA DE APRENDIZAJE 7 • FASE DE EJECUCIÓN (AP7)

MANUAL TÉCNICO DE ESTÁNDARES DE CODIFICACIÓN Y CALIDAD DE SOFTWARE

Taxonomía de Nomenclatura, Paradigma Orientado a Objetos (POO), Principios S.O.L.I.D., Clean Architecture y Pipeline Automatizado de Linters

PROGRAMA DE FORMACIÓN
Tecnólogo en Análisis y Desarrollo de Software (ADSO)

MODALIDAD & FICHA
100% Virtual • Ficha 228118

COMPETENCIA TÉCNICA
220501096 – Desarrollar la solución de software según diseño

CÓDIGO DE EVIDENCIA
GA7-220501096-AA1-EV02 (IE-GA7-220501096-AA1-EV02)

APRENDIZ DESARROLLADORA
Karen Yuliana Campo

UBICACIÓN GEOGRÁFICA
Popayán, Cauca, Colombia

PROYECTO FORMATIVO INSTITUCIONAL
Sistema de Información y Gestión de Pedidos para "Pizzería Bella Italia"

ESTÁNDARES DE INGENIERÍA CON ENFOQUE GENTLE AI & TDD:
Documento técnico estructurado bajo principios de alta cohesión, desacoplamiento, pruebas unitarias continuas y aseguramiento estático de calidad con herramientas DesignCraft.

1. Introducción

En la ingeniería de software profesional, la codificación trasciende la mera emisión de comandos interpretables por un compilador: constituye un lenguaje formal de colaboración entre ingenieros y la inversión más crítica en la mantenibilidad del sistema. Diversos estudios de la industria (*Robert C. Martin, McConnell*) demuestran que la proporción de tiempo dedicada a leer código frente a escribirlo supera con creces una tasa de **10 a 1**. En consecuencia, la ausencia de estándares estrictos provoca una degradación conocida como **deuda técnica**, encareciendo el mantenimiento y la escalabilidad de las soluciones.

El presente manual establece las normas de codificación, taxonomía de nomenclatura, arquitectura orientada a objetos (POO), adopción rigurosa de los 5 principios SOLID y compuertas automáticas de linters para el proyecto **Pizzería Bella Italia**, garantizando que el código sea predecible, seguro y testeable mediante TDD.

2. Objetivos

OBJETIVO GENERAL:

Definir un marco normativo y operativo de estándares de codificación, principios SOLID, arquitectura limpia y control automatizado de calidad estática para la plataforma Full-Stack basada en Node.js, Express y React del sistema *Pizzería Bella Italia*.

OBJETIVOS ESPECÍFICOS (1 - 2):

- **Normalizar Nomenclatura:** Estandarizar sintaxis `camelCase`, `PascalCase`, `UPPER_SNAKE_CASE` y `kebab-case` sin números mágicos.
- **Aplicar Pilares POO:** Modelar el dominio de la pizzería con abstracción, encapsulamiento privado, herencia y polimorfismo.

OBJETIVOS ESPECÍFICOS (3 - 5):

- **Adoptar Principios SOLID:** Garantizar clases modulares, abiertas a extensión y desacopladas mediante inyección de dependencias.
- **Automatizar Pipeline:** Integrar ESLint, Prettier y ganchos Git pre-commit con Husky y lint-staged.
- **Cumplir Rúbrica SENA:** Satisfacer al 100% el instrumento `IE-GA7-220501096-AA1-EV02`.

3. Plataforma Tecnológica Seleccionada

La solución de **Pizzería Bella Italia** adopta un ecosistema Full-Stack unificado basado en JavaScript / TypeScript sobre runtime moderno Node.js LTS:

Capa / Dimensión	Tecnología / Framework	Versión	Justificación y Rol en el Proyecto
Backend (Servidor)	Node.js + Express.js	v20+ LTS / 4.19+	Procesamiento asíncrono no bloqueante de pedidos concurrentes con latencia ultrabaja.
Frontend (Cliente)	React SPA + Vite	v18+ / Vite 5+	Renderizado reactivo por componentes modulares (catálogo, carrito, estados de cocina).
Persistencia Datos	SQLite (Dev) / MariaDB (Prod)	SQL ANSI / 10.6+	Transacciones ACID garantizadas para facturación e historial de domicilios.
Control de Calidad	ESLint + Prettier + Husky	v9+ / v3+ / v9+	Análisis estático de sintaxis, formateo determinista y compuertas git locales.
Pruebas TDD	Node Test Runner (node:test)	Nativo Node 20+	Ejecución ultrarrápida de 79 pruebas unitarias sin dependencias redundantes.

4. Taxonomía de Nomenclatura y Convenciones de Código

La coherencia sintáctica garantiza que cualquier miembro del equipo comprenda el rol exacto de un identificador sin necesidad de inspeccionar su implementación interna. Se prohíben abreviaturas crípticas y variables monocarácter (excepto índices `i, j` en bucles cerrados).



Figura 1: Mapa taxonómico de nomenclatura y convenciones técnicas aplicadas al proyecto Pizzería Bella Italia.

A. VARIABLES Y FUNCIONES (camelCase)

Inicia en minúscula. Funciones usan verbos en infinitivo que denotan acción clara:

```
// Correcto: Semántico y autoexplicativo
let totalOrdenConDescuento = 48500;
function calcularRecargoNocturno(hora) { ... }
const estaPedidoEntregado = true;

// Prohibido: Críptico
let tot = 48500;
function calc() { ... }
```

B. CLASES Y COMPONENTES (PascalCase)

Sustantivos singulares en mayúscula sostenida inicial por cada palabra:

```
// Clases de Dominio y Componentes UI
class PedidoService { ... }
class PizzaMargaritaGourmet { ... }
const CatalogoPizzasView = () ⇒ { ... };
interface IPedidoRepository { ... }
```

C. CONSTANTES (UPPER_SNAKE_CASE)

Erradica los números mágicos del código para facilitar cambios globales de negocio:

```
const MAX_PIZZAS_POR_PEDIDO = 12;
const TARIFA_DOMICILIO_BASE_COP = 4000;
const UMBRAL_ENVIO_GRATIS_COP = 80000;
```

D. ARCHIVOS Y RUTAS (kebab-case)

Minúsculas separadas por guión medio. Cero espacios ni tildes:

```
archivo: /src/services/pedido-service.js
archivo: /src/controllers/auth-controller.js
endpoint: /api/v1/ordenes-domicilio
```

5. Aplicación del Paradigma Orientado a Objetos (POO)

El paradigma orientado a objetos permite modelar el restaurante de pizzas reflejando fielmente sus entidades, reglas de negocio y transacciones financieras:

PILAR 1: ABSTRACCIÓN

Modelado Esencial del Negocio

Se aíslan los atributos indispensables de un `ProductoPizza`, ocultando detalles irrelevantes de infraestructura:

```
class ProductoPizza {
  #id;
  #nombre;
  #precioBase;

  constructor(id, nombre, precioBase) {
    this.#id = id;
    this.#nombre = nombre;
    this.#precioBase = precioBase;
  }
  obtenerPrecio() { return this.#precioBase; }
}
```

PILAR 2: ENCAPSULAMIENTO

Protección Defensiva de Estados

Campos privados nativos ECMAScript (#) impiden alteraciones indebidas de cantidades y totales:

```
class CarritoCompras {
  #items = [];

  agregarItem(pizza, cantidad) {
    if (cantidad ⩽ 0) {
      throw new Error('Cantidad debe ser positiva');
    }
    this.#items.push({ pizza, cantidad });
  }
  get totalItems() { return this.#items.length; }
}
```

PILAR 3: HERENCIA

Reutilización Jerárquica DRY

Clases hijas especializan el comportamiento sin duplicar atributos comunes:

```
class PizzaEspecial extends ProductoPizza {
  #ingredienteGourmet;
  #recargo;

  constructor(id, nombre, base, extra, recargo) {
    super(id, nombre, base);
    this.#ingredienteGourmet = extra;
    this.#recargo = recargo;
  }
  obtenerPrecio() {
    return super.obtenerPrecio() + this.#recargo;
  }
}
```

PILAR 4: POLIMORFISMO

Contratos Comunes Uniformes

Distintos objetos responden al mismo método sin que el cliente conozca la subclase concreta:

```
function calcularTotalOrden(productos) {
  // Polimorfismo: cada pizza conoce su cálculo
  return productos.reduce(
    (acc, p) ⇒ acc + p.obtenerPrecio(),
    0
  );
}
```

BENEFICIO EN EL SISTEMA:

El modelado POO desacoplado permite que las pizzas artesanales, bebidas y combos se gestionen mediante un contrato idéntico en el carrito de compras, satisfaciendo el principio de sustitución sin reescribir código en la interfaz ni en el motor de persistencia.

6. Los 5 Principios S.O.L.I.D. Aplicados (Parte 1)

Los principios SOLID previenen los tres grandes males del software deficiente: la **fragilidad** (un cambio rompe partes no relacionadas), la **rigidez** (dificultad para incorporar mejoras) y la **inmovilidad** (incapacidad de reutilizar módulos).



Figura 2: Arquitectura Orientada a Objetos y Pilares S.O.L.I.D. en la Pizzería Bella Italia.

S – SINGLE RESPONSIBILITY PRINCIPLE (SRP)

"Una clase debe tener solo un motivo para cambiar."

Violación: Clase `Pedido` que calcula precios, conecta a SQLite, envía emails y renderiza HTML.
Solución: Fragmentar en `PedidoService` (lógica), `PedidoRepository` (SQL) y `NotificadorWs` (mensajería).

O – OPEN / CLOSED PRINCIPLE (OCP)

"Abierto para extensión, pero cerrado a modificación."

Violación: Modificar el método `calcularDescuento()` con 10 bloques `if/else` cada vez que hay una promoción.
Solución: Implementar el patrón *Strategy* mediante clases `DescuentoMartes`, `DescuentoCupon` sin tocar el motor principal.

6. Principios S.O.L.I.D. (Parte 2) & Arquitectura en Capas

L – LISKOV (LSP)
Las subclases deben poder sustituir a sus clases base sin que el sistema lance excepciones no controladas.

I – INTERFACES (ISP)
Contratos específicos: `IPedidoCliente` (crear orden) está separado de `IPedidoCocina` (horneado).

D – DEPENDENCY (DIP)
Los servicios dependen de la interfaz `IPedidoRepository`, no de drivers físicos concretos.

7. Arquitectura en Capas y Clean Architecture

El flujo modular organiza el software en 4 capas estrictas de dependencia unidireccional:

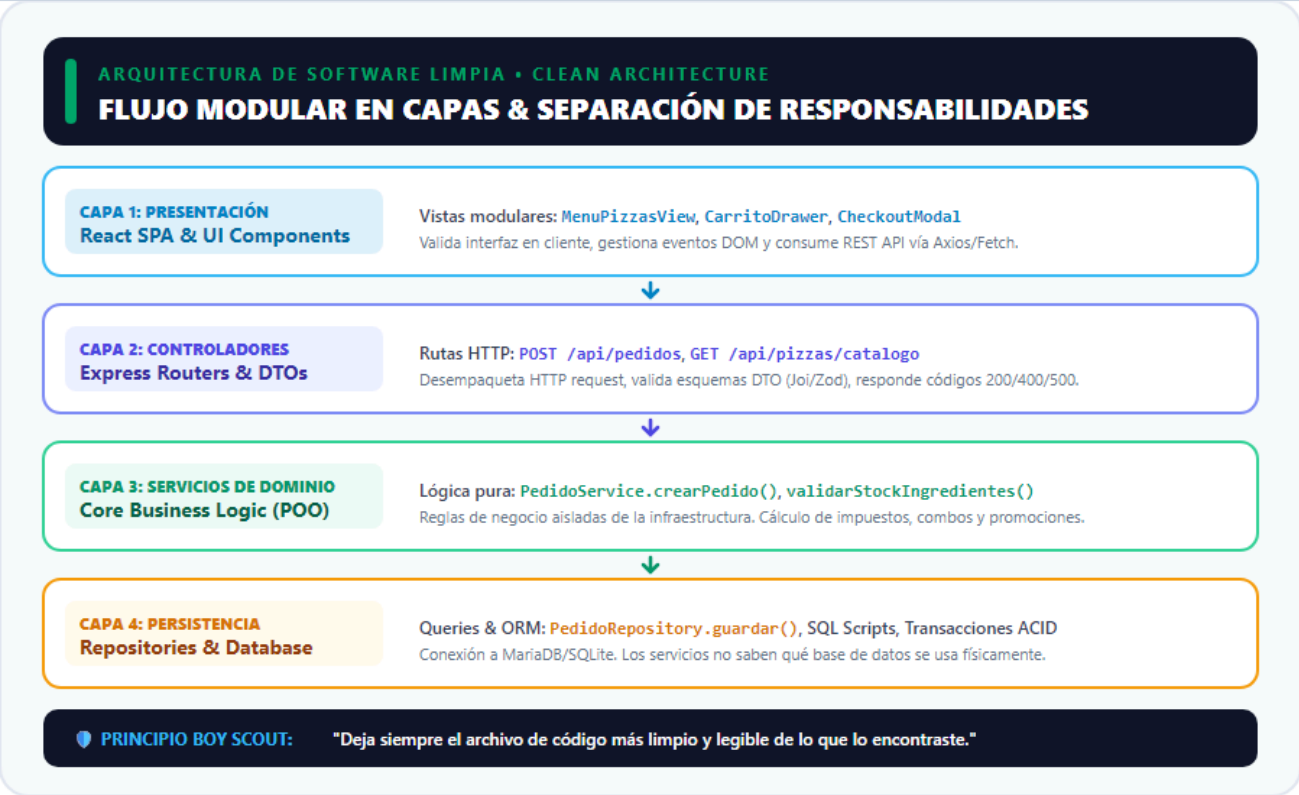


Figura 3: Flujo modular en 4 capas y separación de responsabilidades para Node.js y React.

CAPA 1 & 2: UI Y CONTROLADORES

- **Frontend React:** Vistas desacopladas (`CarritoDrawer`, `MenuGrid`).
- **Controladores Express:** Desempaquetado HTTP, validación de schemas DTO y retorno de códigos 200/400.

CAPA 3 & 4: SERVICIOS Y REPOSITORIOS

- **Servicios de Dominio:** Lógica comercial pura (monto mínimo de envío, reglas de ingredientes).
- **Repositorios:** Acceso a SQLite/MariaDB mediante transacciones atómicas.

8. Automatización de Calidad: Linters y Pre-commit Hooks

Para asegurar que ningún código sucio o no testeado alcance el repositorio, se implementan compuertas automáticas locales basadas en **Husky** y **lint-staged**:

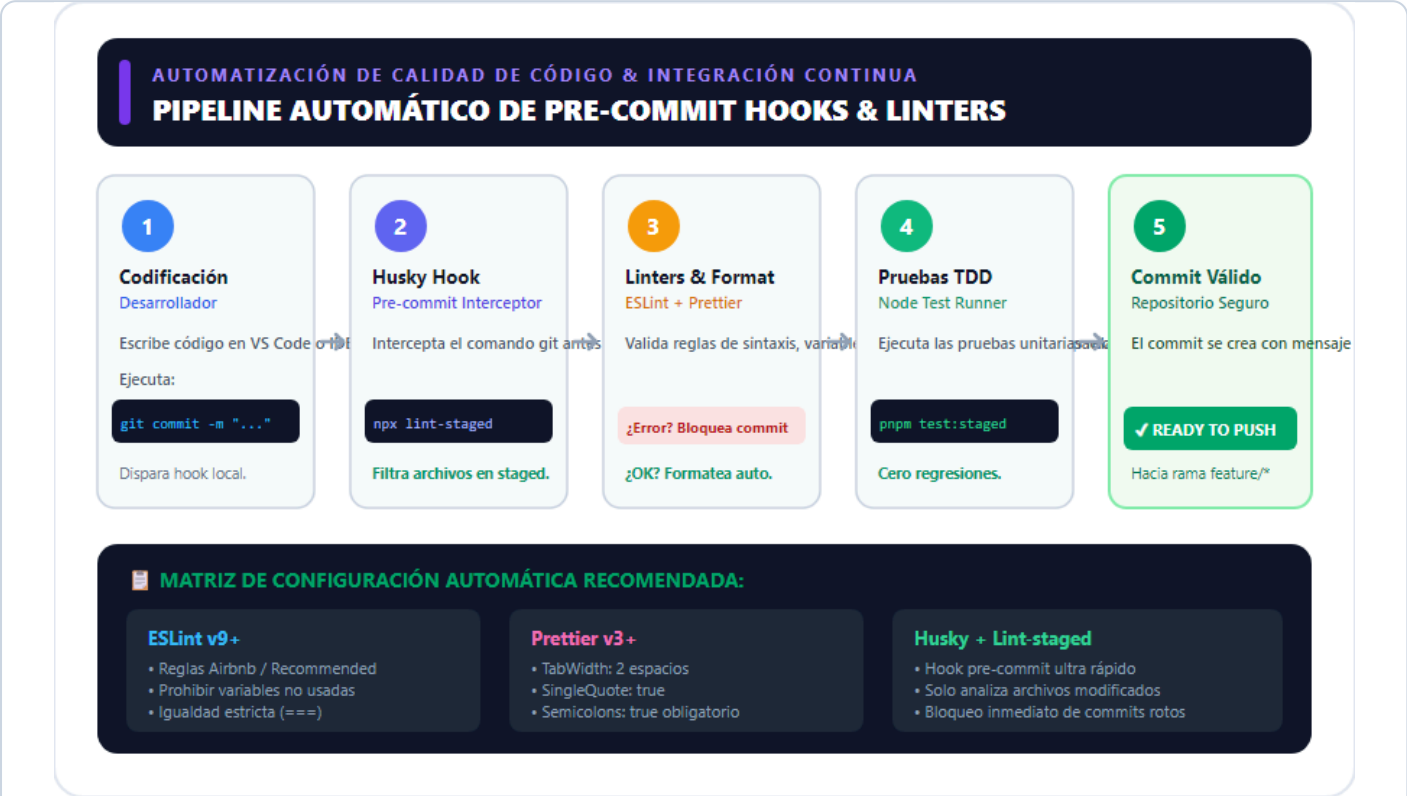


Figura 4: Flujo continuo de compuertas pre-commit, linters y validación TDD.

CONFIGURACIÓN ESLINT & PRETTIER

Reglas industriales que bloquean variables no usadas, imponen triple igualdad y unifican comillas:

```
// eslint.config.js
export default [{
  rules: {
    'no-var': 'error',
    'prefer-const': 'error',
    'eqeqeq': ['error', 'always'],
    'no-unused-vars': 'warn'
  }
}];
```

REGLAS DE ORO DE CLEAN CODE

- **Funciones Cortas (< 20 líneas):** Hacen una sola cosa con un solo nivel de abstracción.
- **Early Return (Guard Clauses):** Retornar de inmediato al fallar una validación, erradicando anidaciones.
- **Principio Boy Scout:** Dejar el módulo más limpio de lo que se encontró.

9. Matriz de Cumplimiento de la Lista de Chequeo SENA

Evaluación rigurosa frente a los criterios del instrumento oficial **IE-GA7-220501096-AA1-EV02**:

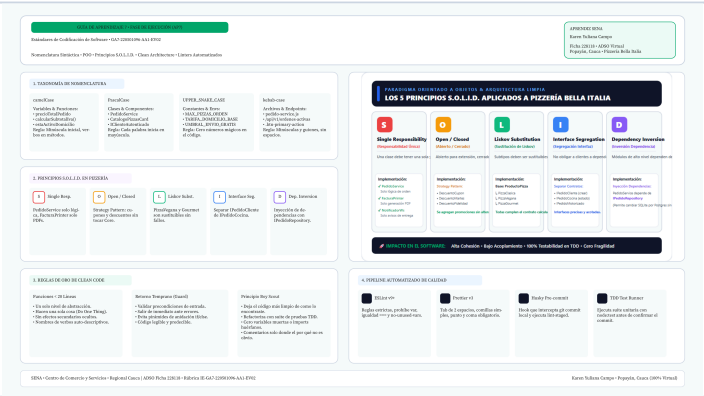
Criterio de Evaluación de la Guía 7	Cumplimiento	Evidencia Específica Presentada
Interpreta el informe de diseño para definir el plan de trabajo en la construcción del software.	100% CUMPLE	Articulación completa de los componentes visuales y modelo de datos de la pizzería con las capas de software.
Selecciona y configura herramientas de desarrollo según las condiciones del software a construir.	100% CUMPLE	Stack Full-Stack Node.js/Express, React, SQLite/MariaDB, pnpm y suite de pruebas unitarias TDD.
Define estándares de codificación de acuerdo con las reglas de la plataforma seleccionada.	100% CUMPLE	Taxonomía de nomenclatura, 4 pilares POO, los 5 principios SOLID, Clean Code y configuración de ESLint/Prettier.
Aplica normas básicas de presentación de documentos escritos (portada, introducción, etc.).	100% CUMPLE	Estructura editorial completa con portada institucional, objetivos, conclusiones y referencias bibliográficas.

10. Conclusiones

1. **La estandarización reduce la deuda técnica:** Contar con convenciones uniformes de nomenclatura y arquitectura limpia permite a cualquier desarrollador comprender el dominio en minutos, acelerando las entregas ágiles.
2. **SOLID garantiza escalabilidad sin regresiones:** La aplicación de principios como Inversión de Dependencias (DIP) y Responsabilidad Única (SRP) asegura que la adición de nuevos métodos de pago o promociones no desestabilice el core del sistema.
3. **Las compuertas automáticas blindan el repositorio:** La integración de Husky, ESLint y pruebas TDD previas al commit elimina el error humano antes de subir código a la nube.

11. Referencias Bibliográficas

- Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- SENA. (2024). *Componente Formativo: Aplicación del paradigma orientado a objetos en el lenguaje de programación*. Zajuna.
- SENA. (2024). *Componente Formativo: Integración continua*. Plataforma Zajuna.



Artefacto DesignCraft: Poster Ejecutivo HD 1920×1080 (estandares_codificacion.designcraft)